

MANUEL PROFESSIONNEL ET UNIVERSITAIRE

Consultant en innovation technologique, transformation numérique, cybersécurité, intelligence artificielle, développement logiciel et stratégie digitale.

ARCHITECTIDE

Contact

Site web

Coordonnées

www.charmantnyungu.com

Email

consultant@charmantnyungu.com

Téléphone

+243 835 137 837

Édition

Formation complète — 2026

Charmant Nyungu

Vision du livre

Comprendre

Structurer

Tester

Sécuriser

Déployer

Ce support privilégie les visuels, les schémas, les exemples de code, les tableaux de décision et les exercices guidés.

Présentation du manuel

Ce manuel d'architecture logicielle est conçu pour apprendre à organiser un vrai projet logiciel de manière professionnelle. Il s'adresse aux étudiants, développeurs, chefs de projets, entrepreneurs, ingénieurs, architectes débutants et responsables techniques qui veulent comprendre comment passer d'un code fonctionnel à un système maintenable, sécurisé et évolutif.

L'auteur, Charmant Nyungu, y propose une approche progressive : comprendre les responsabilités, découper le système, documenter les décisions, tester les comportements, sécuriser les flux et préparer l'exploitation en production.

Élément	Détail
Auteur	Charmant Nyungu
Profession	Consultant en innovation technologique, transformation numérique, cybersécurité, intelligence artificielle, développement logiciel et
Site	www.charmantnyungu.com
Email	consultant@charmantnyungu.com
Téléphone	+243 835 137 837

Préface

L'architecture logicielle n'est pas une décoration ajoutée à la fin du développement. Elle est la manière dont une équipe transforme une ambition en système durable. Une bonne architecture permet de comprendre vite, modifier sans peur, tester avec confiance et livrer avec discipline.

Un projet mal organisé peut fonctionner pendant quelques semaines, puis devenir coûteux à maintenir. Un projet bien structuré donne à l'équipe une mémoire, une direction et une capacité d'évolution. Ce livre montre comment atteindre cet objectif avec des méthodes concrètes.

Comment utiliser ce livre

- Lire chaque chapitre avec un carnet de décisions techniques.
- Reproduire les diagrammes à partir de vos propres projets.
- Exécuter les exemples de code et les modifier volontairement.
- Comparer plusieurs architectures avant de choisir.
- Écrire une documentation courte pour chaque décision importante.

Table des matières détaillée

1. Partie 1 : Fondations de l'architecture logicielle

- Rôle de l'architecte logiciel : vision, arbitrage, responsabilité, communication
- Penser système : frontière, flux, dépendance, contrat
- Qualités non fonctionnelles : performance, sécurité, maintenabilité, observabilité

2. Partie 2 : Organisation d'un vrai projet

- Arborescence professionnelle : dossiers, noms, conventions, lisibilité
- Modules et responsabilités : cohésion, couplage, interfaces, réutilisation
- Services applicatifs : cas d'usage, orchestration, transactions, erreurs

3. Partie 3 : Modèles d'architecture

- MVC et variantes modernes : modèle, vue, contrôleur, présentation
- Architecture en couches : UI, application, domaine, infrastructure
- Architecture hexagonale : ports, adaptateurs, domaine, tests
- Microservices et monolithe modulaire : frontières, déploiement, données, complexité

4. Partie 4 : Clean code et conception

- Nommage et intention : variables, fonctions, classes, langage métier
- SOLID en pratique : SRP, OCP, LSP, ISP, DIP
- Design patterns utiles : Factory, Strategy, Repository, Observer
- Refactoring : odeurs de code, petits pas, tests, revue

5. Partie 5 : Tests et qualité

- Pyramide de tests : unitaires, intégration, end-to-end, contrats
- Tests automatisés : fixtures, mocks, CI, couverture
- Qualité continue : lint, formatage, analyse statique, revue

6. Partie 6 : Documentation et gouvernance

- Documentation vivante : README, ADR, diagrammes, runbooks
- Cahier d'architecture : contexte, contraintes, décisions, risques
- Communication technique : public, niveau, preuve, synthèse

7. Partie 7 : Sécurité architecturale

- Security by design : menaces, surface, contrôle, journalisation
- Authentification et autorisation : identité, rôles, permissions, sessions
- Protection des données : chiffrement, secrets, sauvegardes, rétention
- Audit et conformité : logs, traçabilité, incidents, preuve

8. Partie 8 : API, intégration et données

- Conception d'API : REST, contrats, versioning, erreurs
- Bases de données et architecture : modèle, index, transactions, migration
- Événements et files de messages : asynchrone, résilience, ordre, idempotence

9. Partie 9 : Déploiement, observabilité et exploitation

- CI/CD : pipeline, validation, release, rollback

- Monitoring : métriques, logs, traces, alertes
- Scalabilité et performance : cache, files, profiling, capacité

10. Partie 10 : Projets professionnels complets

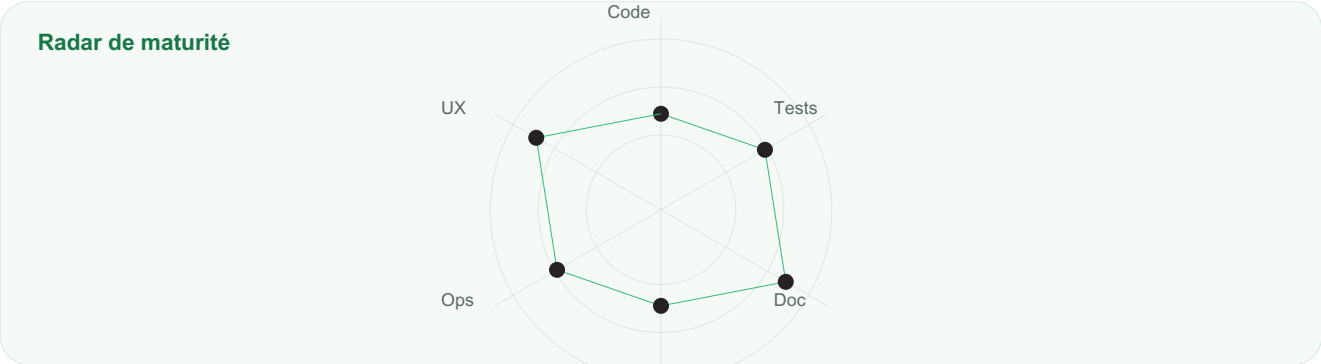
- Architecture d'une plateforme e-commerce : catalogue, panier, paiement, logistique
- Architecture d'un système hospitalier : patients, rendez-vous, dossiers, confidentialité
- Architecture d'un système financier : wallet, transactions, audit, fraude
- Architecture d'une application éducative : cours, examens, certificats, analytique
- Architecture d'une super app africaine : modules, identité, paiement, sécurité

11. Partie 11 : Exercices, certification et annexes

- Banque d'exercices : analyse, diagrammes, code, revue
- Examens blancs : questions, cas, projet, soutenance
- Glossaire et feuilles de route : termes, références, progression, seniorité

Rôle de l'architecte logiciel

Ce chapitre explique rôle de l'architecte logiciel dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Rôle de l'architecte logiciel.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
vision	Comment vision influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
arbitrage	Comment arbitrage influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
responsabilité	Comment responsabilité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
communication	Comment communication influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

vision

vision doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, vision aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : vision**arbitrage**

arbitrage doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, arbitrage aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

responsabilité

responsabilité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, responsabilité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : responsabilité**communication**

communication doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, communication aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Rôle de l'architecte logiciel

```
class visionService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si rôle de l'architecte logiciel est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment vision doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment arbitrage doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment responsabilité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment communication doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment vision doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

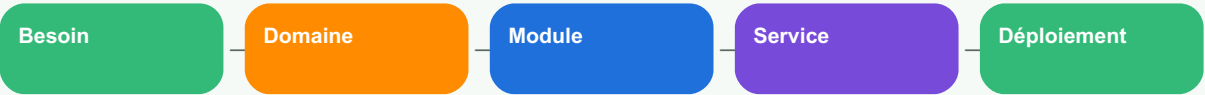
Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Penser système

Ce chapitre explique penser système dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Flux architectural



Objectifs opérationnels

- Identifier les responsabilités associées à Penser système.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
frontière	Comment frontière influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
flux	Comment flux influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
dépendance	Comment dépendance influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
contrat	Comment contrat influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

frontière

frontière doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, frontière aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : frontière**flux**

flux doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, flux aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

dépendance

dépendance doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, dépendance aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : dépendance**contrat**

contrat doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, contrat aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Penser système

```
class frontiereService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si penser système est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment frontière doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment flux doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment dépendance doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment contrat doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment frontière doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Qualités non fonctionnelles

Ce chapitre explique qualités non fonctionnelles dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches

Interface utilisateur

Application

Domaine métier

Infrastructure

Données

Objectifs opérationnels

- Identifier les responsabilités associées à Qualités non fonctionnelles.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
performance	Comment performance influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
sécurité	Comment sécurité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
maintenabilité	Comment maintenabilité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
observabilité	Comment observabilité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

performance

performance doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, performance aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : performance**sécurité**

sécurité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, sécurité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

maintenabilité

maintenabilité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, maintenabilité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : maintenabilité**observabilité**

observabilité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, observabilité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Qualités non fonctionnelles

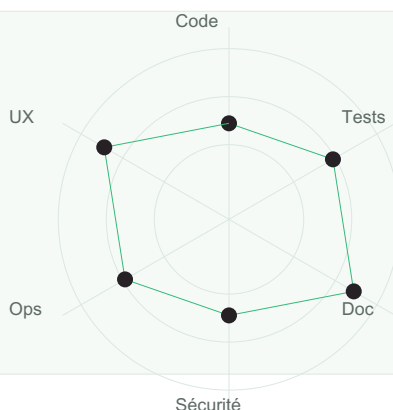
```
class performanceService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si qualités non fonctionnelles est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment performance doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment sécurité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment maintenabilité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment observabilité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment performance doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

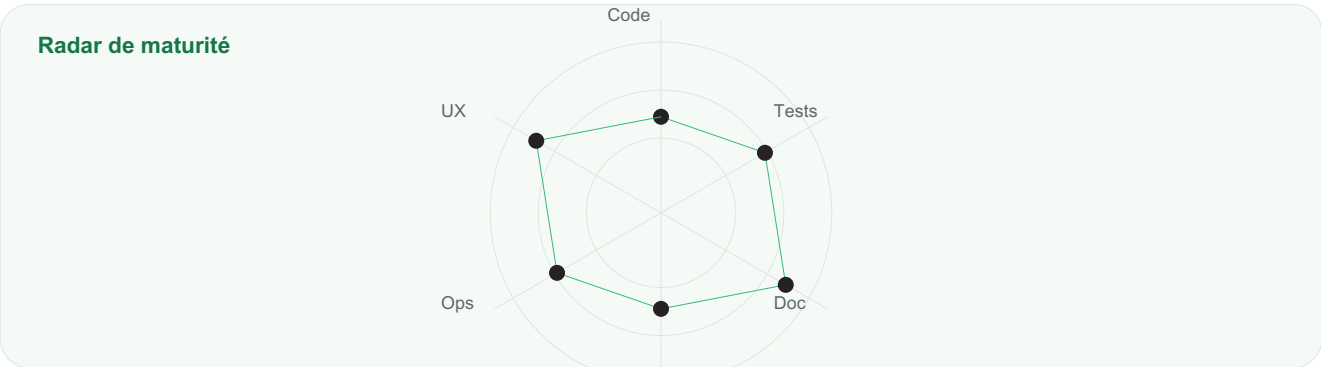
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Arborescence professionnelle

Ce chapitre explique arborescence professionnelle dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Arborescence professionnelle.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
dossiers	Comment dossiers influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
noms	Comment noms influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
conventions	Comment conventions influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
lisibilité	Comment lisibilité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

dossiers

dossiers doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, dossiers aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : dossiers**noms**

noms doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, noms aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

conventions

conventions doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, conventions aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : conventions**lisibilité**

lisibilité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, lisibilité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Arborescence professionnelle

```
class dossiersService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si arborescence professionnelle est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment dossiers doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment noms doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment conventions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment lisibilité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment dossiers doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

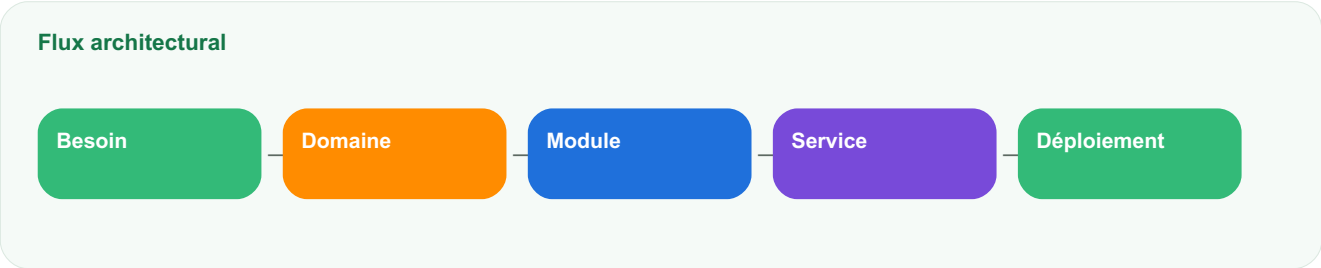
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Modules et responsabilités

Ce chapitre explique modules et responsabilités dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Modules et responsabilités.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
cohésion	Comment cohésion influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
couplage	Comment couplage influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
interfaces	Comment interfaces influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
réutilisation	Comment réutilisation influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

cohésion

cohésion doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, cohésion aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : cohésion**couplage**

couplage doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, couplage aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

interfaces

interfaces doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, interfaces aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : interfaces**réutilisation**

réutilisation doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, réutilisation aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Modules et responsabilités

```
class cohSIONService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si modules et responsabilités est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment cohésion doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment couplage doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment interfaces doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment réutilisation doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment cohésion doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Services applicatifs

Ce chapitre explique services applicatifs dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches



Objectifs opérationnels

- Identifier les responsabilités associées à Services applicatifs.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
cas d'usage	Comment cas d'usage influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
orchestration	Comment orchestration influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
transactions	Comment transactions influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
erreurs	Comment erreurs influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

cas d'usage

cas d'usage doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, cas d'usage aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : cas d'usage**orchestration**

orchestration doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, orchestration aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

transactions

transactions doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, transactions aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : transactions**erreurs**

erreurs doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, erreurs aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Services applicatifs

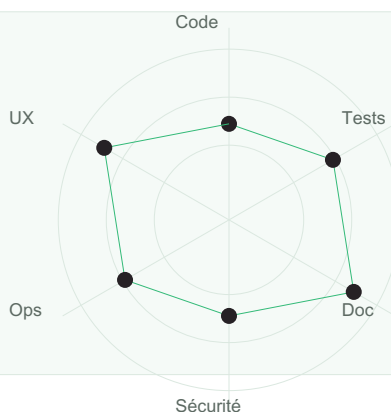
```
class casdusageService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si services applicatifs est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment cas d'usage doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment orchestration doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment transactions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment erreurs doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment cas d'usage doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

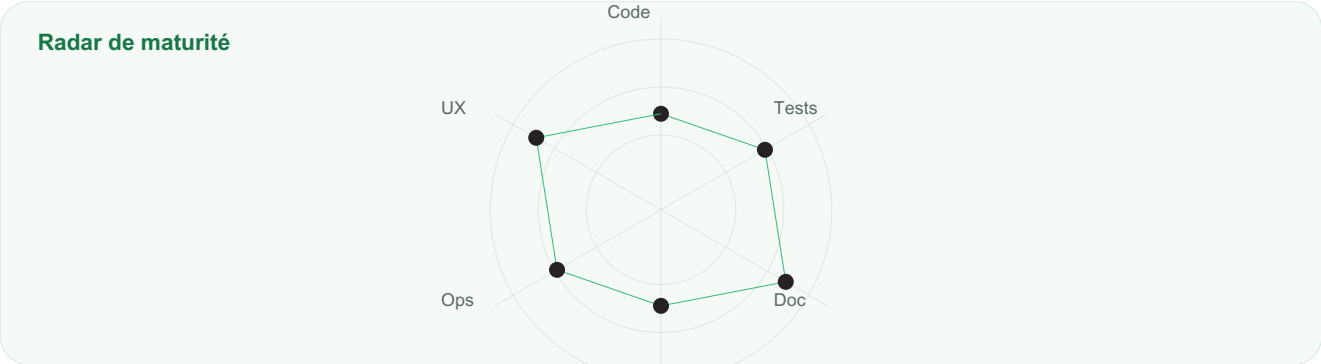
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

MVC et variantes modernes

Ce chapitre explique mvc et variantes modernes dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à MVC et variantes modernes.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
modèle	Comment modèle influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
vue	Comment vue influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
contrôleur	Comment contrôleur influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
présentation	Comment présentation influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

modèle

modèle doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, modèle aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : modèle**vue**

vue doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, vue aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

contrôleur

contrôleur doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, contrôleur aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : contrôleur**présentation**

présentation doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, présentation aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — MVC et variantes modernes

```
class UserModel:
    def __init__(self, name):
        self.name = name

class UserView:
    def render(self, user):
        return f"Utilisateur : {user.name}"

class UserController:
    def __init__(self, view):
        self.view = view

    def show_profile(self, name):
        user = UserModel(name)
        return self.view.render(user)

print(UserController(UserView()).show_profile("Amina"))
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si mvc et variantes modernes est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.

- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment modèle doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment vue doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment contrôleur doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment présentation doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment modèle doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

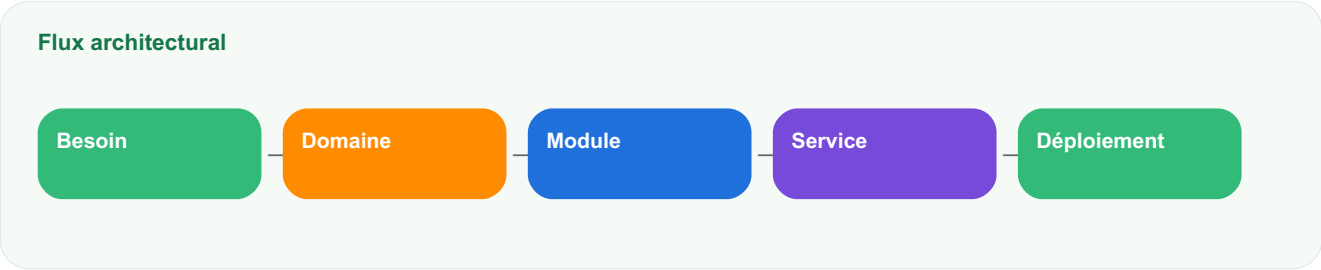
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Architecture en couches

Ce chapitre explique architecture en couches dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Architecture en couches.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
UI	Comment UI influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
application	Comment application influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
domaine	Comment domaine influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
infrastructure	Comment infrastructure influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

UI

UI doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, ui aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : UI**application**

application doit être comprise comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, application aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

domaine

domaine doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, domaine aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : domaine**infrastructure**

infrastructure doit être comprise comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, infrastructure aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Architecture en couches

```
class UIService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si architecture en couches est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment UI doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment application doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment domaine doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment infrastructure doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment UI doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Architecture hexagonale

Ce chapitre explique architecture hexagonale dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches

Interface utilisateur

Application

Domaine métier

Infrastructure

Données

Objectifs opérationnels

- Identifier les responsabilités associées à Architecture hexagonale.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
ports	Comment ports influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
adaptateurs	Comment adaptateurs influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
domaine	Comment domaine influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
tests	Comment tests influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

ports

ports doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, ports aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : ports**adaptateurs**

adaptateurs doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, adaptateurs aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

domaine

domaine doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, domaine aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : domaine**tests**

tests doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, tests aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Architecture hexagonale

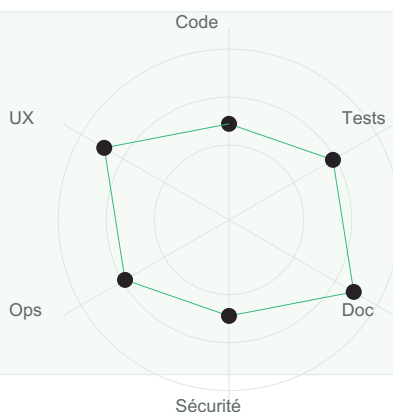
```
class portsService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si architecture hexagonale est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment ports doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment adaptateurs doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment domaine doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment tests doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment ports doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Microservices et monolithe modulaire

Ce chapitre explique microservices et monolithe modulaire dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Matrice de risque technique



Matrice impact × probabilité pour prioriser les risques techniques.

Objectifs opérationnels

- Identifier les responsabilités associées à Microservices et monolithe modulaire.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
frontières	Comment frontières influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
déploiement	Comment déploiement influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
données	Comment données influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
complexité	Comment complexité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

frontières

frontières doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, frontières aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : frontières**déploiement**

déploiement doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, déploiement aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

données

données doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, données aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : données**complexité**

complexité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, complexité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Microservices et monolithe modulaire

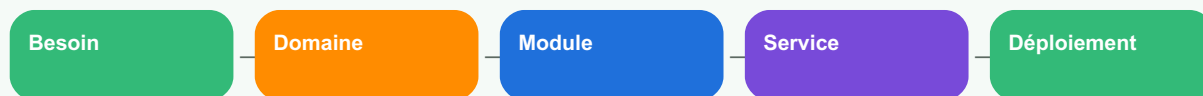
```
class frontieresService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si microservices et monolithe modulaire est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Flux architectural

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment frontières doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment déploiement doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment données doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment complexité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment frontières doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

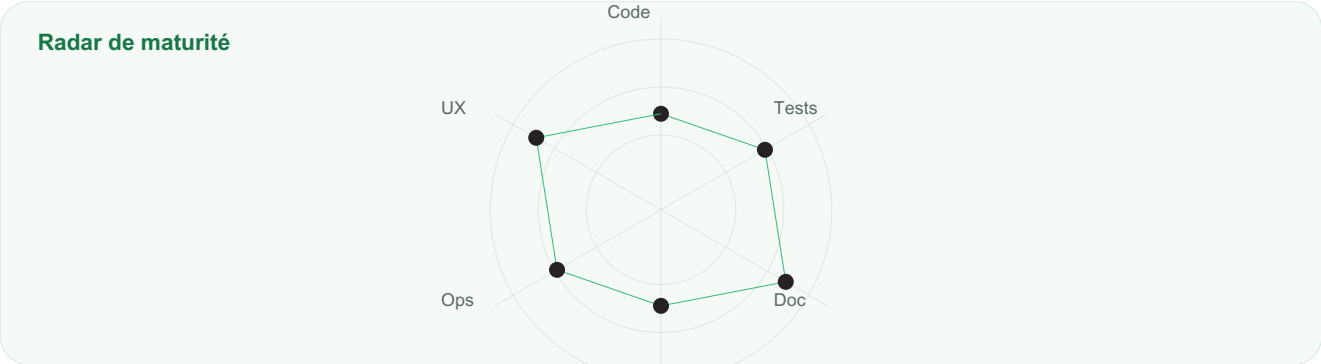
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Nommage et intention

Ce chapitre explique nommage et intention dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Nommage et intention.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
variables	Comment variables influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
fonctions	Comment fonctions influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
classes	Comment classes influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
langage métier	Comment langage métier influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

variables

variables doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, variables aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : variables**fonctions**

fonctions doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, fonctions aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

classes

classes doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, classes aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : classes**langage métier**

langage métier doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, langage métier aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Nommage et intention

```
class variablesService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si nommage et intention est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment variables doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment fonctions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment classes doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment langage métier doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment variables doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

SOLID en pratique

Ce chapitre explique solid en pratique dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches

Interface utilisateur

Application

Domaine métier

Infrastructure

Données

Objectifs opérationnels

- Identifier les responsabilités associées à SOLID en pratique.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
SRP	Comment SRP influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
OCP	Comment OCP influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
LSP	Comment LSP influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
ISP	Comment ISP influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
DIP	Comment DIP influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

SRP

SRP doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, srp aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : SRP



OCP

OCP doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, ocp aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

LSP

LSP doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, lsp aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : LSP



ISP

ISP doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, isp aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

DIP

DIP doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, dip aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : DIP



Exemple de code professionnel

Code — SOLID en pratique

```

class SRPService:
    def __init__(self, repository):
        self.repository = repository

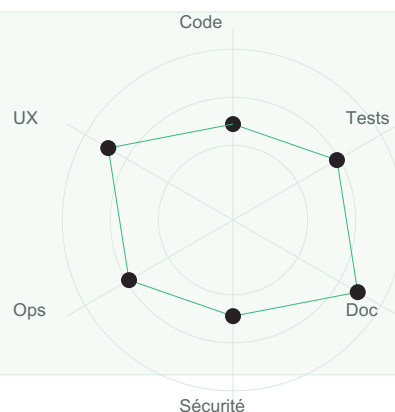
    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
  
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si solid en pratique est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité



Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment SRP doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment OCP doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment LSP doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment ISP doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment DIP doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?

- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Design patterns utiles

Ce chapitre explique design patterns utiles dans une logique professionnelle. L’objectif n’est pas seulement de connaître une définition, mais de savoir l’appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches



Objectifs opérationnels

- Identifier les responsabilités associées à Design patterns utiles.
- Découper le système en éléments compréhensibles par l’équipe.
- Écrire du code dont l’intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d’architecture	Décision attendue
Factory	Comment Factory influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
Strategy	Comment Strategy influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
Repository	Comment Repository influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
Observer	Comment Observer influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

Factory

Factory doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d’architecture n’a de valeur que si elle réduit l’ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n’est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d’accès et des règles de maintenance. De la même manière, factory aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : Factory**Strategy**

Strategy doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, strategy aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Repository

Repository doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, repository aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : Repository**Observer**

Observer doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, observer aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Design patterns utiles

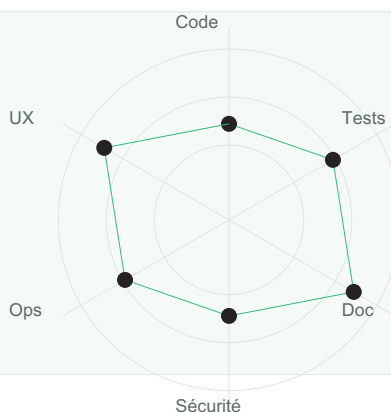
```
class FactoryService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si design patterns utiles est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment Factory doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment Strategy doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment Repository doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment Observer doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment Factory doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Refactoring

Ce chapitre explique refactoring dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Objectifs opérationnels

- Identifier les responsabilités associées à Refactoring.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
odeurs de code	Comment odeurs de code influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
petits pas	Comment petits pas influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
tests	Comment tests influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
revue	Comment revue influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

odeurs de code

odeurs de code doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, odeurs de code aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : odeurs de code**petits pas**

petits pas doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, petits pas aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

tests

tests doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, tests aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : tests**revue**

revue doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, revue aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Refactoring

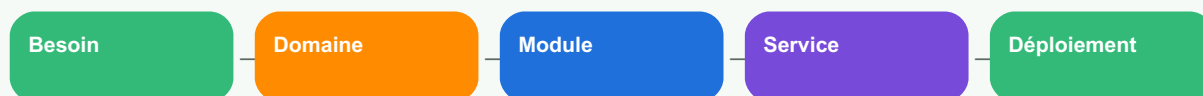
```
class odeursdecodeService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si refactoring est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Flux architectural

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment odeurs de code doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment petits pas doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment tests doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment revue doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment odeurs de code doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

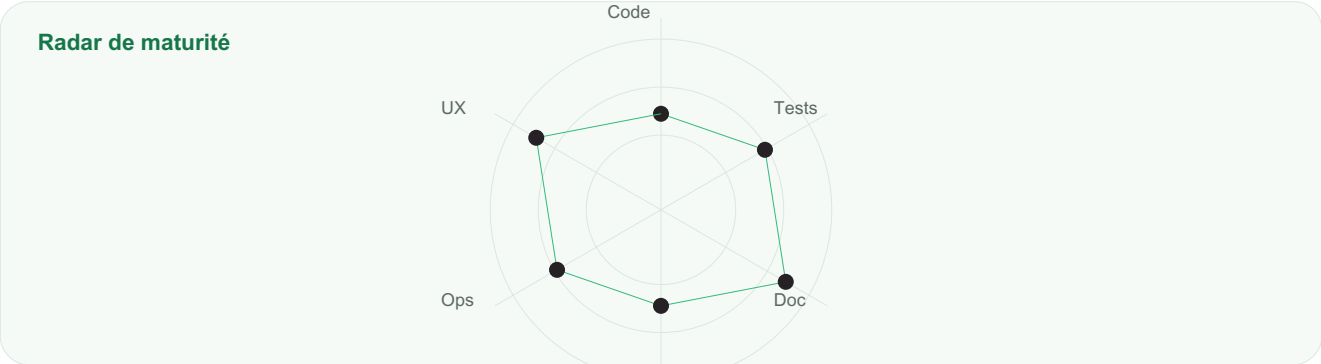
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Pyramide de tests

Ce chapitre explique pyramide de tests dans une logique professionnelle. L’objectif n’est pas seulement de connaître une définition, mais de savoir l’appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Pyramide de tests.
- Découper le système en éléments compréhensibles par l’équipe.
- Écrire du code dont l’intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d’architecture	Décision attendue
unitaires	Comment unitaires influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
intégration	Comment intégration influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
end-to-end	Comment end-to-end influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
contrats	Comment contrats influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

unitaires

unitaires doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d’architecture n’a de valeur que si elle réduit l’ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n’est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d’accès et des règles de maintenance. De la même manière, unitaires aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : unitaires**intégration**

intégration doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, intégration aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

end-to-end

end-to-end doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, end-to-end aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : end-to-end**contrats**

contrats doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, contrats aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Pyramide de tests

```
def calculer_total(prix, taxe):
    if prix < 0:
        raise ValueError("Le prix ne peut pas être négatif")
    return round(prix + prix * taxe, 2)

def test_calculer_total():
    assert calculer_total(100, 0.16) == 116.0
    assert calculer_total(0, 0.16) == 0.0
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si pyramide de tests est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible

Faible / Moyen

Faible / Élevé

Moyen / Faible

Moyen / Moyen

Moyen / Élevé

Élevé / Faible

Élevé / Moyen

Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment unitaires doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment intégration doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment end-to-end doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment contrats doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment unitaires doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

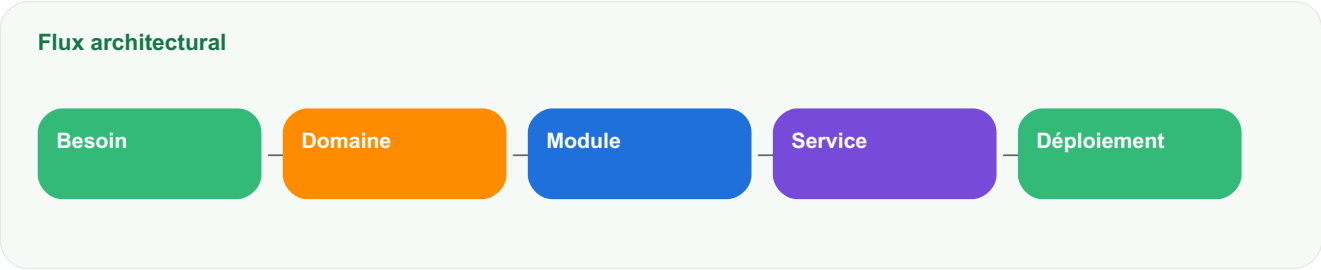
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Tests automatisés

Ce chapitre explique tests automatisés dans une logique professionnelle. L’objectif n’est pas seulement de connaître une définition, mais de savoir l’appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Tests automatisés.
- Découper le système en éléments compréhensibles par l’équipe.
- Écrire du code dont l’intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d’architecture	Décision attendue
fixtures	Comment fixtures influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
mocks	Comment mocks influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
CI	Comment CI influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
couverture	Comment couverture influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

fixtures

fixtures doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d’architecture n’a de valeur que si elle réduit l’ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n’est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d’accès et des règles de maintenance. De la même manière, fixtures aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : fixtures**mocks**

mocks doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, mocks aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

CI

CI doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, ci aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : CI**couverture**

couverture doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, couverture aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Tests automatisés

```
def calculer_total(prix, taxe):
    if prix < 0:
        raise ValueError("Le prix ne peut pas être négatif")
    return round(prix + prix * taxe, 2)

def test_calculer_total():
    assert calculer_total(100, 0.16) == 116.0
    assert calculer_total(0, 0.16) == 0.0
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si tests automatisés est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment fixtures doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment mocks doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment CI doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment couverture doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment fixtures doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Qualité continue

Ce chapitre explique qualité continue dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches



Objectifs opérationnels

- Identifier les responsabilités associées à Qualité continue.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
lint	Comment lint influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
formatage	Comment formatage influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
analyse statique	Comment analyse statique influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
revue	Comment revue influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

lint

lint doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, lint aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : lint**formatage**

formatage doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, formatage aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

analyse statique

analyse statique doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, analyse statique aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : analyse statique**revue**

revue doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, revue aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Qualité continue

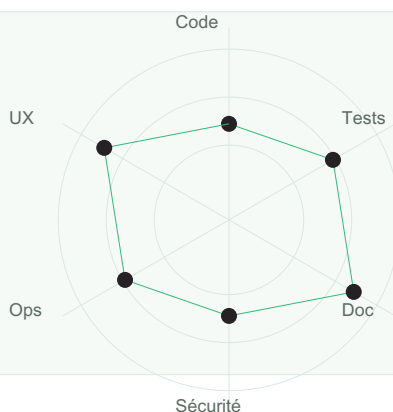
```
class lintService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si qualité continue est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment lint doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment formatage doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment analyse statique doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment revue doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment lint doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

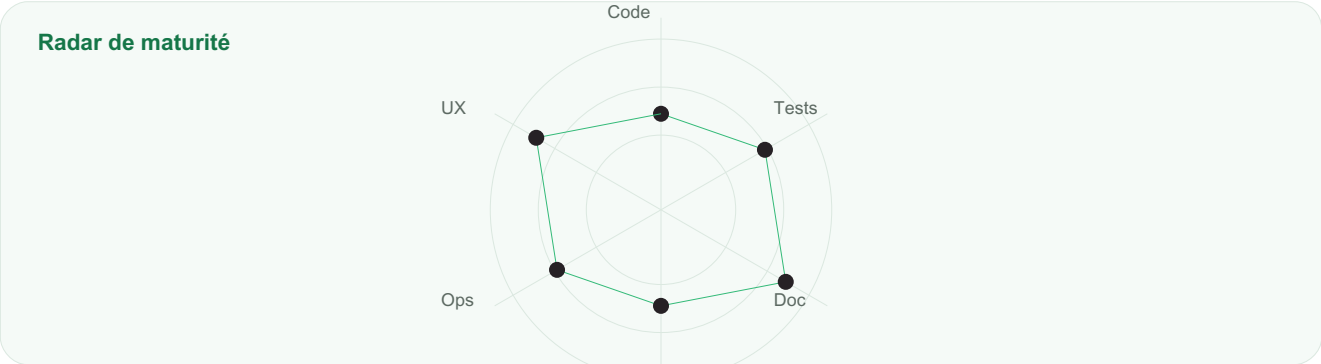
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Documentation vivante

Ce chapitre explique documentation vivante dans une logique professionnelle. L’objectif n’est pas seulement de connaître une définition, mais de savoir l’appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Documentation vivante.
- Découper le système en éléments compréhensibles par l’équipe.
- Écrire du code dont l’intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d’architecture	Décision attendue
README	Comment README influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
ADR	Comment ADR influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
diagrammes	Comment diagrammes influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
runbooks	Comment runbooks influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

README

README doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d’architecture n’a de valeur que si elle réduit l’ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n’est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d’accès et des règles de maintenance. De la même manière, readme aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : README**ADR**

ADR doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, adr aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

diagrammes

diagrammes doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, diagrammes aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : diagrammes**runbooks**

runbooks doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, runbooks aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Documentation vivante

```
class READMEService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si documentation vivante est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment README doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment ADR doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment diagrammes doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment runbooks doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment README doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

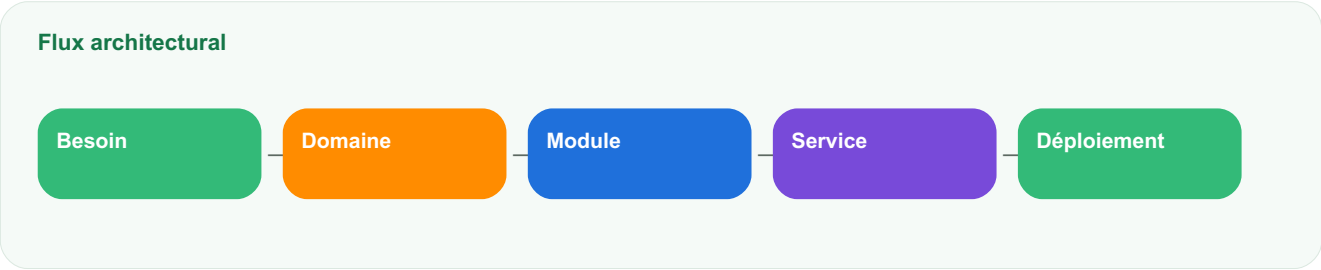
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Cahier d'architecture

Ce chapitre explique cahier d'architecture dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Cahier d'architecture.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
contexte	Comment contexte influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
contraintes	Comment contraintes influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
décisions	Comment décisions influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
risques	Comment risques influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

contexte

contexte doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, contexte aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : contexte**contraintes**

contraintes doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, contraintes aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

décisions

décisions doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, décisions aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : décisions**risques**

risques doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, risques aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Cahier d'architecture

```
class contexteService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si cahier d'architecture est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment contexte doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment contraintes doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment décisions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment risques doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment contexte doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Communication technique

Ce chapitre explique communication technique dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches



Objectifs opérationnels

- Identifier les responsabilités associées à Communication technique.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
public	Comment public influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
niveau	Comment niveau influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
preuve	Comment preuve influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
synthèse	Comment synthèse influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

public

public doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, public aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : public**niveau**

niveau doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, niveau aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

preuve

preuve doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, preuve aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : preuve**synthèse**

synthèse doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, synthèse aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Communication technique

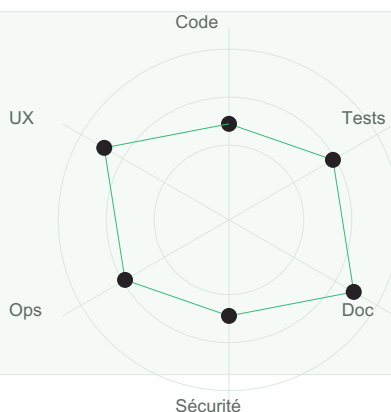
```
class publicService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si communication technique est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment public doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment niveau doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment preuve doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment synthèse doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment public doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

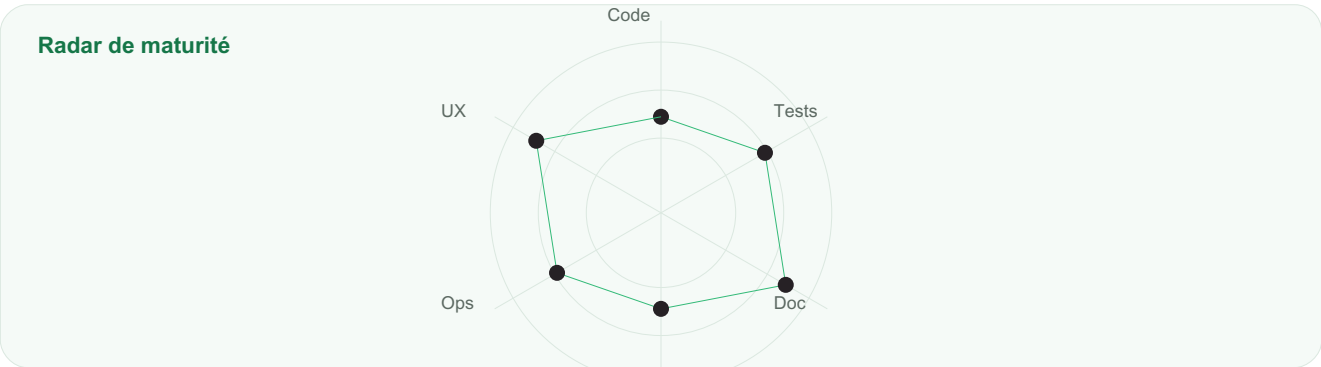
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Security by design

Ce chapitre explique security by design dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Security by design.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
menaces	Comment menaces influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
surface	Comment surface influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
contrôle	Comment contrôle influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
journalisation	Comment journalisation influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

menaces

menaces doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, menaces aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : menaces**surface**

surface doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, surface aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

contrôle

contrôle doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, contrôle aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : contrôle**journalisation**

journalisation doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, journalisation aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Security by design

```
class menacesService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si security by design est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment menaces doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment surface doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment contrôle doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment journalisation doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment menaces doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

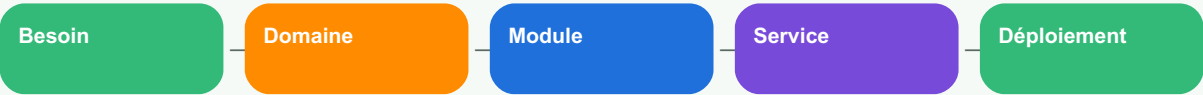
Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Authentification et autorisation

Ce chapitre explique authentification et autorisation dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Flux architectural



Objectifs opérationnels

- Identifier les responsabilités associées à Authentification et autorisation.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
identité	Comment identité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
rôles	Comment rôles influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
permissions	Comment permissions influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
sessions	Comment sessions influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

identité

identité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, identité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : identité**rôles**

rôles doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, rôles aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

permissions

permissions doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, permissions aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : permissions**sessions**

sessions doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, sessions aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Authentification et autorisation

```
def can_access(user, resource):
    if not user.get("active"):
        return False
    permissions = user.get("permissions", [])
    return resource in permissions

agent = {"active": True, "permissions": ["orders:read", "payments:read"]}
print(can_access(agent, "orders:read"))
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si authentification et autorisation est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment identité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment rôles doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment permissions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment sessions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment identité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Protection des données

Ce chapitre explique protection des données dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches



Objectifs opérationnels

- Identifier les responsabilités associées à Protection des données.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
chiffrement	Comment chiffrement influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
secrets	Comment secrets influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
sauvegardes	Comment sauvegardes influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
rétenion	Comment rétenion influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

chiffrement

chiffrement doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, chiffrement aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : chiffrement**secrets**

secrets doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, secrets aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

sauvegards

sauvegards doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, sauvegards aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : sauvegards**rétenion**

rétenion doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, rétenion aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Protection des données

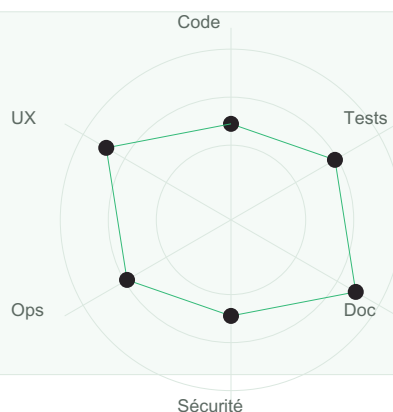
```
class chiffrementService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si protection des données est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment chiffrement doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment secrets doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment sauvegardes doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment rétention doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment chiffrement doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Audit et conformité

Ce chapitre explique audit et conformité dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Matrice de risque technique



Matrice impact × probabilité pour prioriser les risques techniques.

Objectifs opérationnels

- Identifier les responsabilités associées à Audit et conformité.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
logs	Comment logs influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
traçabilité	Comment traçabilité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
incidents	Comment incidents influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
preuve	Comment preuve influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

logs

logs doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, logs aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : logs**traçabilité**

traçabilité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, traçabilité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

incidents

incidents doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, incidents aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : incidents**preuve**

preuve doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, preuve aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Audit et conformité

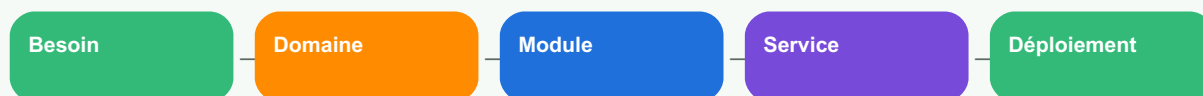
```
class logsService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si audit et conformité est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Flux architectural

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment logs doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment traçabilité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment incidents doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment preuve doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment logs doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

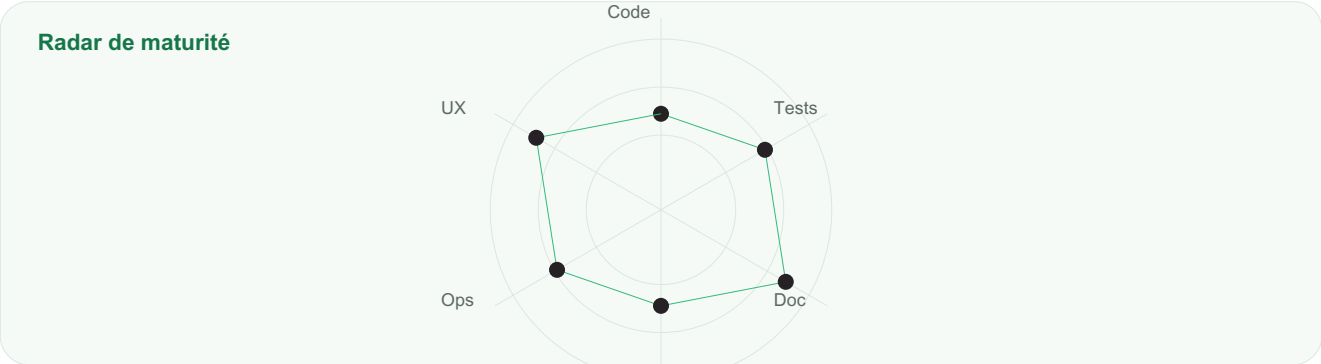
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Conception d'API

Ce chapitre explique conception d'api dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Conception d'API.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
REST	Comment REST influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
contrats	Comment contrats influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
versioning	Comment versioning influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
erreurs	Comment erreurs influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

REST

REST doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, rest aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : REST**contrats**

contrats doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, contrats aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

versioning

versioning doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, versioning aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : versioning**erreurs**

erreurs doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, erreurs aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Conception d'API

```
from fastapi import FastAPI, HTTPException

app = FastAPI()
orders = {"CMD-001": {"status": "paid"}}

@app.get("/orders/{reference}")
def get_order(reference: str):
    if reference not in orders:
        raise HTTPException(status_code=404, detail="Commande introuvable")
    return {"reference": reference, **orders[reference]}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si conception d'api est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment REST doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment contrats doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment versioning doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment erreurs doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment REST doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

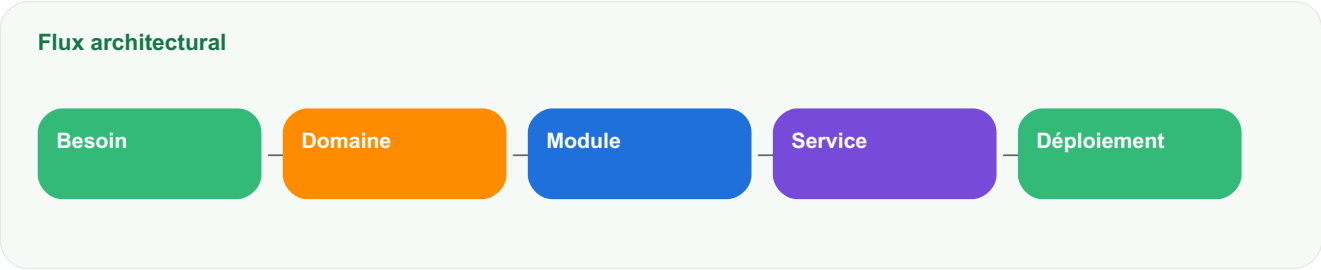
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Bases de données et architecture

Ce chapitre explique bases de données et architecture dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Bases de données et architecture.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
modèle	Comment modèle influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
index	Comment index influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
transactions	Comment transactions influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
migration	Comment migration influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

modèle

modèle doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, modèle aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : modèle**index**

index doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, index aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

transactions

transactions doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, transactions aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : transactions**migration**

migration doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, migration aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Bases de données et architecture

```
class modleService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si bases de données et architecture est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment modèle doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment index doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment transactions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment migration doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment modèle doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Événements et files de messages

Ce chapitre explique événements et files de messages dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches



Objectifs opérationnels

- Identifier les responsabilités associées à Événements et files de messages.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
asynchrone	Comment asynchrone influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
résilience	Comment résilience influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
ordre	Comment ordre influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
idempotence	Comment idempotence influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

asynchrone

asynchrone doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, asynchrone aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : asynchrone**résilience**

résilience doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, résilience aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

ordre

ordre doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, ordre aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : ordre**idempotence**

idempotence doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, idempotence aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Événements et files de messages

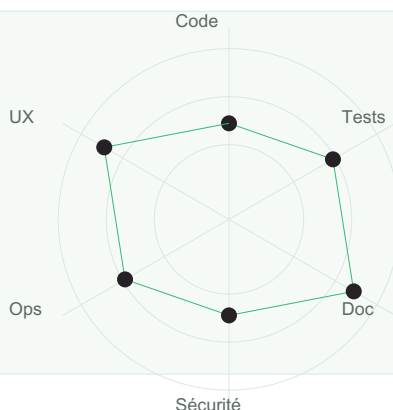
```
class asynchroneService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si événements et files de messages est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment asynchrone doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment résilience doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment ordre doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment idempotence doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment asynchrone doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

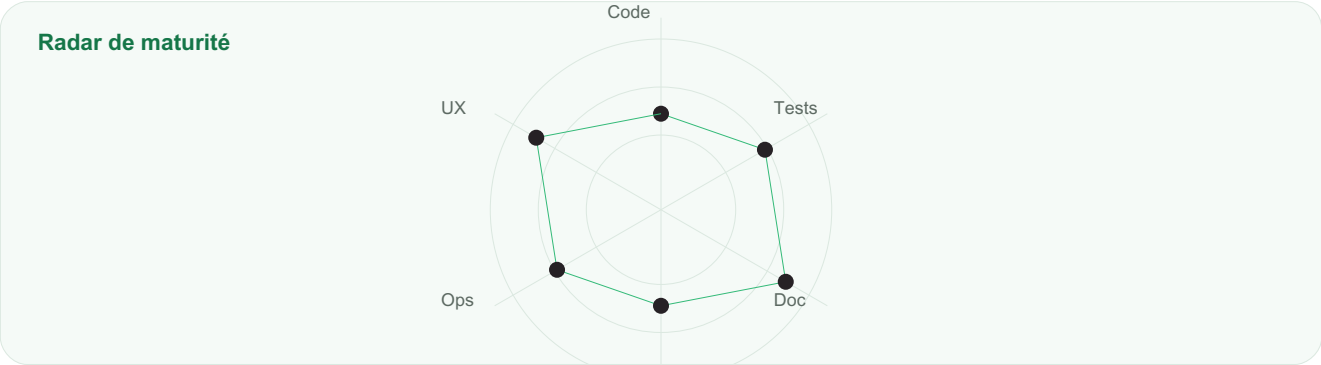
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

CI/CD

Ce chapitre explique ci/cd dans une logique professionnelle. L’objectif n’est pas seulement de connaître une définition, mais de savoir l’appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à CI/CD.
- Découper le système en éléments compréhensibles par l’équipe.
- Écrire du code dont l’intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d’architecture	Décision attendue
pipeline	Comment pipeline influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
validation	Comment validation influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
release	Comment release influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
rollback	Comment rollback influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

pipeline

pipeline doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d’architecture n’a de valeur que si elle réduit l’ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n’est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d’accès et des règles de maintenance. De la même manière, pipeline aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : pipeline**validation**

validation doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, validation aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

release

release doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, release aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : release**rollback**

rollback doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, rollback aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — CI/CD

```
class pipelineService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si ci/cd est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment pipeline doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment validation doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment release doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment rollback doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment pipeline doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

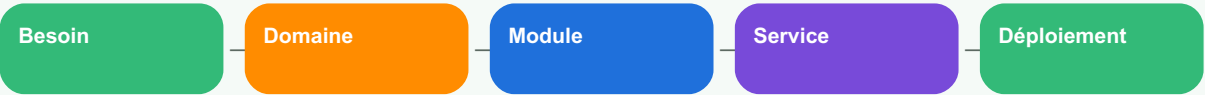
Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Monitoring

Ce chapitre explique monitoring dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Flux architectural



Objectifs opérationnels

- Identifier les responsabilités associées à Monitoring.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
métriques	Comment métriques influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
logs	Comment logs influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
traces	Comment traces influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
alertes	Comment alertes influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

métriques

métriques doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, métriques aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : métriques**logs**

logs doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, logs aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

traces

traces doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, traces aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : traces**alertes**

alertes doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, alertes aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Monitoring

```
class mtriquesService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si monitoring est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment métriques doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment logs doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment traces doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment alertes doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment métriques doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Scalabilité et performance

Ce chapitre explique scalabilité et performance dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches

Interface utilisateur

Application

Domaine métier

Infrastructure

Données

Objectifs opérationnels

- Identifier les responsabilités associées à Scalabilité et performance.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
cache	Comment cache influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
files	Comment files influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
profiling	Comment profiling influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
capacité	Comment capacité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

cache

cache doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, cache aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : cache**files**

files doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, files aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

profiling

profiling doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, profiling aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : profiling**capacité**

capacité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, capacité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Scalabilité et performance

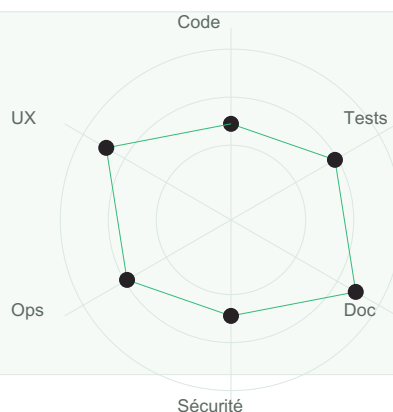
```
class cacheService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si scalabilité et performance est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment cache doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment files doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment profiling doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment capacité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment cache doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

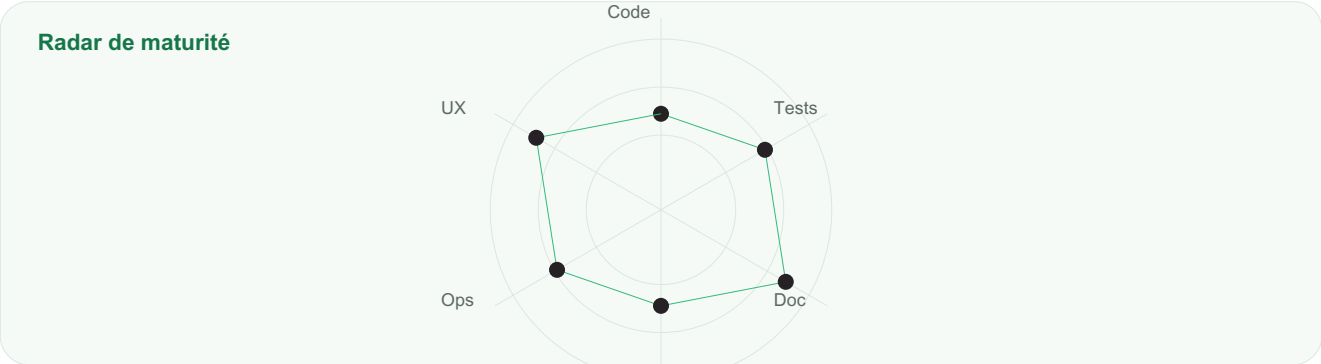
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Architecture d’une plateforme e-commerce

Ce chapitre explique architecture d’une plateforme e-commerce dans une logique professionnelle. L’objectif n’est pas seulement de connaître une définition, mais de savoir l’appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Architecture d’une plateforme e-commerce.
- Découper le système en éléments compréhensibles par l’équipe.
- Écrire du code dont l’intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d’architecture	Décision attendue
catalogue	Comment catalogue influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
panier	Comment panier influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
paiement	Comment paiement influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
logistique	Comment logistique influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

catalogue

catalogue doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d’architecture n’a de valeur que si elle réduit l’ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n’est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d’accès et des règles de maintenance. De la même manière, catalogue aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : catalogue**panier**

panier doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, panier aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

paiement

paiement doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, paiement aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : paiement**logistique**

logistique doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, logistique aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Architecture d'une plateforme e-commerce

```
class catalogueService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si architecture d'une plateforme e-commerce est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment catalogue doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment panier doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment paiement doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment logistique doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment catalogue doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

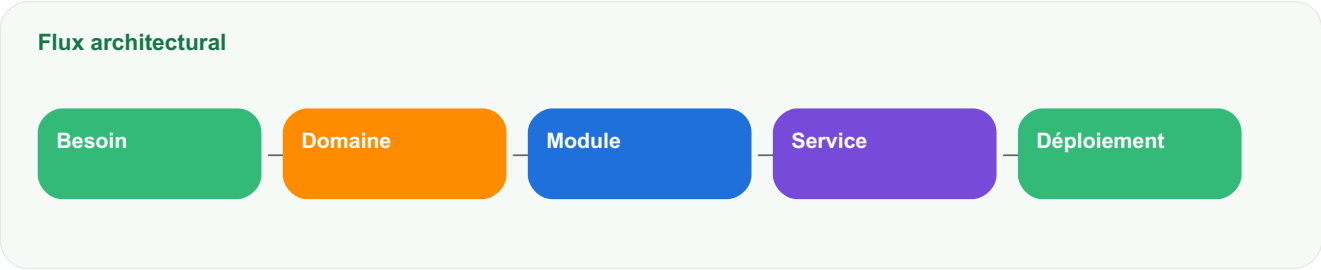
Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Architecture d'un système hospitalier

Ce chapitre explique architecture d'un système hospitalier dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.



Objectifs opérationnels

- Identifier les responsabilités associées à Architecture d'un système hospitalier.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
patients	Comment patients influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
rendez-vous	Comment rendez-vous influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
dossiers	Comment dossiers influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
confidentialité	Comment confidentialité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

patients

patients doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, patients aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : patients**rendez-vous**

rendez-vous doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, rendez-vous aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

dossiers

dossiers doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, dossiers aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : dossiers**confidentialité**

confidentialité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, confidentialité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Architecture d'un système hospitalier

```
class patientsService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si architecture d'un système hospitalier est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment patients doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment rendez-vous doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment dossiers doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment confidentialité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment patients doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Architecture d'un système financier

Ce chapitre explique architecture d'un système financier dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches



Objectifs opérationnels

- Identifier les responsabilités associées à Architecture d'un système financier.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
wallet	Comment wallet influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
transactions	Comment transactions influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
audit	Comment audit influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
fraude	Comment fraude influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

wallet

wallet doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, wallet aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : wallet**transactions**

transactions doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, transactions aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

audit

audit doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, audit aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : audit**fraude**

fraude doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, fraude aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Architecture d'un système financier

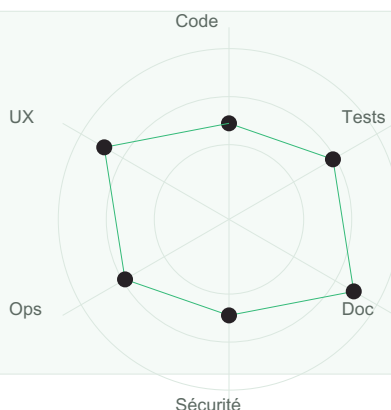
```
class walletService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si architecture d'un système financier est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment wallet doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment transactions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment audit doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment fraude doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment wallet doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Architecture d’une application éducative

Ce chapitre explique architecture d’une application éducative dans une logique professionnelle. L’objectif n’est pas seulement de connaître une définition, mais de savoir l’appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Matrice de risque technique



Matrice impact × probabilité pour prioriser les risques techniques.

Objectifs opérationnels

- Identifier les responsabilités associées à Architecture d’une application éducative.
- Découper le système en éléments compréhensibles par l’équipe.
- Écrire du code dont l’intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d’architecture	Décision attendue
cours	Comment cours influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
examens	Comment examens influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
certificats	Comment certificats influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
analytique	Comment analytique influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

cours

cours doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d’architecture n’a de valeur que si elle réduit l’ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n’est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d’accès et des règles de maintenance. De la même manière, cours aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : cours**examens**

examens doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, examens aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

certificats

certificats doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, certificats aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : certificats**analytique**

analytique doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, analytique aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Architecture d'une application éducative

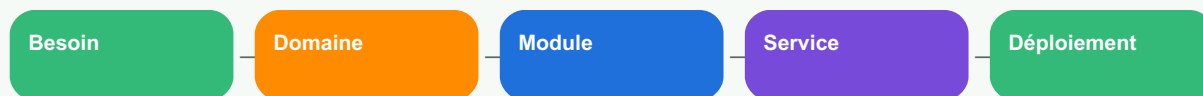
```
class coursService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si architecture d'une application éducative est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Flux architectural

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment cours doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment examens doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment certificats doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment analytique doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment cours doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Architecture d’une super app africaine

Ce chapitre explique architecture d’une super app africaine dans une logique professionnelle. L’objectif n’est pas seulement de connaître une définition, mais de savoir l’appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Comparaison qualitative



Objectifs opérationnels

- Identifier les responsabilités associées à Architecture d’une super app africaine.
- Découper le système en éléments compréhensibles par l’équipe.
- Écrire du code dont l’intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d’architecture	Décision attendue
modules	Comment modules influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
identité	Comment identité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
paiement	Comment paiement influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
sécurité	Comment sécurité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

modules

modules doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d’architecture n’a de valeur que si elle réduit l’ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n’est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d’accès et des règles de maintenance. De la même manière, modules aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : modules**identité**

identité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, identité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

paiement

paiement doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, paiement aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : paiement**sécurité**

sécurité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, sécurité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Architecture d'une super app africaine

```
class modulesService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si architecture d'une super app africaine est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Architecture en couches

Interface utilisateur

Application

Domaine métier

Infrastructure

Données

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment modules doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment identité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment paiement doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment sécurité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment modules doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

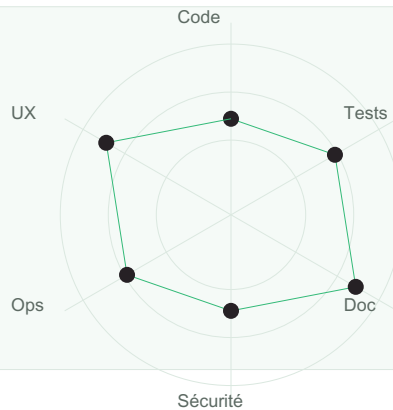
Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Banque d'exercices

Ce chapitre explique banque d'exercices dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Radar de maturité



Objectifs opérationnels

- Identifier les responsabilités associées à Banque d'exercices.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
analyse	Comment analyse influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
diagrammes	Comment diagrammes influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
code	Comment code influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
revue	Comment revue influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

analyse

analyse doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, analyse aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : analyse**diagrammes**

diagrammes doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, diagrammes aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

code

code doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, code aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : code**revue**

revue doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, revue aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Banque d'exercices

```
class analyseService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si banque d'exercices est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment analyse doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment diagrammes doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment code doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment revue doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment analyse doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

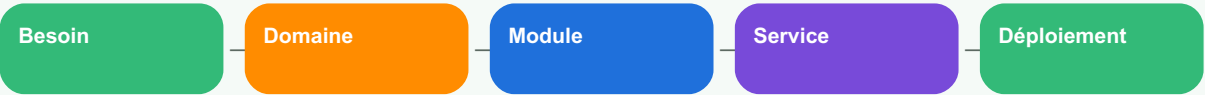
Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Examens blancs

Ce chapitre explique examens blancs dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Flux architectural



Objectifs opérationnels

- Identifier les responsabilités associées à Examens blancs.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
questions	Comment questions influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
cas	Comment cas influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
projet	Comment projet influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
soutenance	Comment soutenance influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

questions

questions doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, questions aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : questions**cas**

cas doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, cas aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

projet

projet doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, projet aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : projet**soutenance**

soutenance doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, soutenance aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Examens blancs

```
class questionsService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si examens blancs est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Comparaison qualitative

Lisibilité



Tests



Sécurité



Docs



Ops

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment questions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment cas doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment projet doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment soutenance doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment questions doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Glossaire et feuilles de route

Ce chapitre explique glossaire et feuilles de route dans une logique professionnelle. L'objectif n'est pas seulement de connaître une définition, mais de savoir l'appliquer dans un vrai projet, avec des modules clairs, des services cohérents, des tests utiles, une documentation maintenable et une sécurité intégrée dès la conception.

Architecture en couches

Interface utilisateur

Application

Domaine métier

Infrastructure

Données

Objectifs opérationnels

- Identifier les responsabilités associées à Glossaire et feuilles de route.
- Découper le système en éléments compréhensibles par l'équipe.
- Écrire du code dont l'intention reste lisible après plusieurs mois.
- Prévoir les tests, la documentation, les erreurs et les risques.
- Relier la décision technique au besoin métier réel.

Concept	Question d'architecture	Décision attendue
termes	Comment termes influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
références	Comment références influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
progression	Comment progression influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.
seniorité	Comment seniorité influence-t-il la structure du projet ?	Définir une règle claire, documentée et testable.

Théorie approfondie

termes

termes doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, termes aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : termes**références**

références doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, références aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

progression

progression doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, progression aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Flux pratique : progression**seniorité**

seniorité doit être compris comme un levier de maîtrise. Dans une équipe professionnelle, une notion d'architecture n'a de valeur que si elle réduit l'ambiguïté, facilite la collaboration et rend le système plus fiable. Le code doit parler le langage du métier, mais il doit aussi respecter les contraintes techniques : sécurité, performance, évolutivité, coûts et exploitation.

Analogie : un bâtiment solide n'est pas seulement une accumulation de briques. Il repose sur des plans, des fondations, des circuits, des zones d'accès et des règles de maintenance. De la même manière, seniorité aide à construire un logiciel qui reste habitable pour les développeurs et fiable pour les utilisateurs.

Exemple de code professionnel

Code — Glossaire et feuilles de route

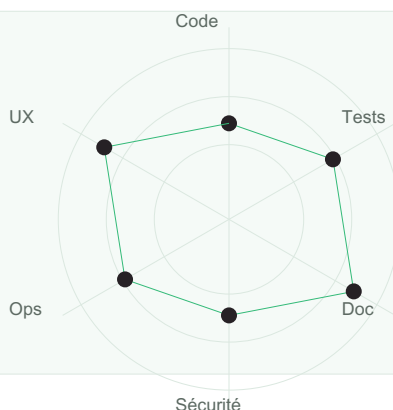
```
class termesService:
    def __init__(self, repository):
        self.repository = repository

    def execute(self, command):
        if not command:
            raise ValueError("Commande invalide")
        result = self.repository.save(command)
        return {"status": "success", "result": result}
```

Le code présenté est volontairement court. En architecture logicielle, la simplicité contrôlée vaut mieux qu'une complexité brillante mais fragile. Le rôle de l'architecte est de rendre le système compréhensible, testable et modifiable.

Étude de cas

Imaginez une plateforme de paiement, de logistique ou d'e-commerce. Si glossaire et feuilles de route est négligé, l'équipe finit par modifier plusieurs fichiers sans savoir où se trouve la vraie règle métier. Si le sujet est bien traité, chaque changement possède un emplacement naturel, une documentation courte et des tests ciblés.

Radar de maturité

Bonnes pratiques

- Documenter les décisions importantes sous forme d'ADR court.
- Éviter les modules fourre-tout qui mélangent plusieurs responsabilités.
- Nommer les services selon les cas d'usage métier.
- Isoler les dépendances externes derrière des interfaces.
- Écrire des tests avant les refactorings risqués.
- Ajouter des logs utiles sans exposer de données sensibles.

Pièges à éviter

- Confondre architecture et empilement de technologies.
- Créer trop de couches sans bénéfice réel.
- Laisser les règles métier dans les composants d'interface.
- Documenter seulement après un incident.
- Croire qu'un microservice corrige un mauvais découpage métier.

Exercices corrigés

Exercice 1 : choisissez un module d'application et décrivez comment termes doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 1 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 2 : choisissez un module d'application et décrivez comment références doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 2 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 3 : choisissez un module d'application et décrivez comment progression doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 3 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 4 : choisissez un module d'application et décrivez comment seniorité doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 4 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Exercice 5 : choisissez un module d'application et décrivez comment termes doit être organisé. Précisez les fichiers, les responsabilités, les tests et les risques.

Corrigé 5 : une réponse solide commence par le besoin métier, propose une frontière claire, nomme les entrées et sorties, prévoit un test unitaire, un test d'intégration et une note de documentation. La solution doit éviter les dépendances cachées.

Quiz de validation

- Quelle responsabilité ce chapitre permet-il de clarifier ?
- Quelle erreur d'organisation peut coûter cher en production ?
- Quel test prouve que la décision est correcte ?
- Quelle information doit apparaître dans la documentation ?
- Quel risque de sécurité ou d'exploitation faut-il surveiller ?

Banque de 420 exercices d'architecture logicielle

Les exercices suivants entraînent l'apprenant à raisonner comme un architecte : analyser, découper, documenter, coder, tester, sécuriser et expliquer. Chaque exercice doit être résolu avec un diagramme et une justification courte.

Modules et services

Architecture en couches

Interface utilisateur

Application

Domaine métier

Infrastructure

Données

Exercice 1 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 2 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 3 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 4 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 5 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 6 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 7 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 8 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 9 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 10 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 27 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 28 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 29 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 30 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 31 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 32 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 33 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 34 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 35 : analysez un problème lié à Modules et services. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

MVC et couches

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Exercice 36 : analysez un problème lié à MVC et couches. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 37 : analysez un problème lié à MVC et couches. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 38 : analysez un problème lié à MVC et couches. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 39 : analysez un problème lié à MVC et couches. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Comparaison qualitative



Lisibilité



Tests



Sécurité



Docs



Ops

Exercice 71 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 72 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 73 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 74 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 75 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 76 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 77 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 78 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 79 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 80 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 81 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 82 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 83 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 100 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 101 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

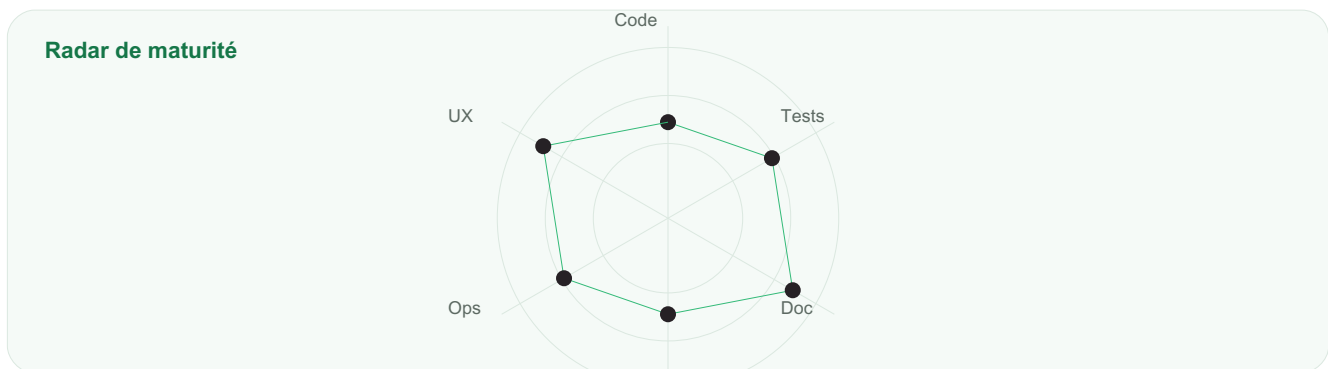
Exercice 102 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 103 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 104 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 105 : analysez un problème lié à Clean code. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Tests



Exercice 106 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 107 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 108 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 109 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 110 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 111 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 112 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 129 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 130 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 131 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 132 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 133 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 134 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 135 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 136 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 137 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

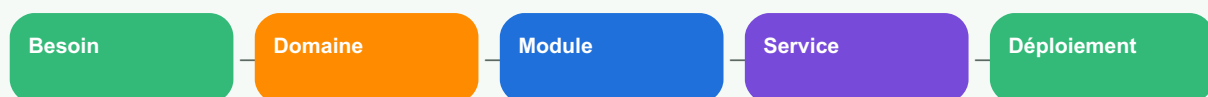
Exercice 138 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 139 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 140 : analysez un problème lié à Tests. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Documentation

Flux architectural



Exercice 141 : analysez un problème lié à Documentation. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 142 : analysez un problème lié à Documentation. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Architecture en couches

Interface utilisateur

Application

Domaine métier

Infrastructure

Données

Exercice 176 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 177 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 178 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 179 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 180 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 181 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 182 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 183 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 184 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 185 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 186 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 187 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 188 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 205 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 206 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 207 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 208 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 209 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 210 : analysez un problème lié à Sécurité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

API et intégration

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Exercice 211 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 212 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 213 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 214 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 215 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 216 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 217 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 234 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 235 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 236 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 237 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 238 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 239 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 240 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 241 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 242 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 243 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 244 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 245 : analysez un problème lié à API et intégration. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Base de données

Comparaison qualitative

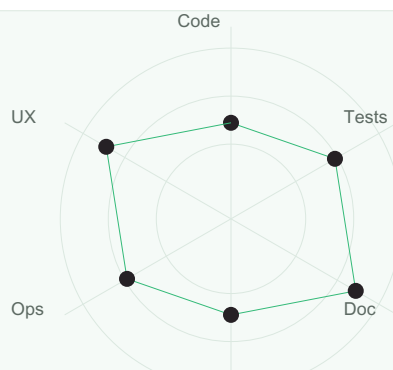


Exercice 246 : analysez un problème lié à Base de données. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 247 : analysez un problème lié à Base de données. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 264 : analysez un problème lié à Base de données. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Radar de maturité



Exercice 281 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 282 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 283 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 284 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 285 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 286 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 287 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 288 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 289 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 290 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 291 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 292 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 293 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 310 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 311 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 312 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

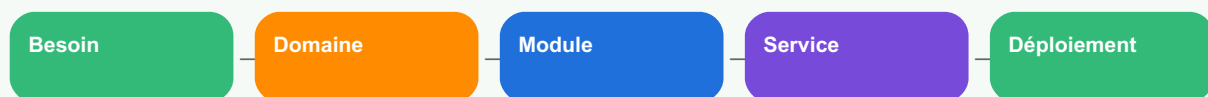
Exercice 313 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 314 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 315 : analysez un problème lié à CI/CD. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Observabilité

Flux architectural



Exercice 316 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 317 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 318 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 319 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 320 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 321 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 322 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 323 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 341 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 342 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 343 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 344 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 345 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 346 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 347 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 348 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 349 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 350 : analysez un problème lié à Observabilité. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Performance

Architecture en couches

Interface utilisateur

Application

Domaine métier

Infrastructure

Données

Exercice 351 : analysez un problème lié à Performance. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 352 : analysez un problème lié à Performance. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Matrice de risque technique

Faible / Faible	Faible / Moyen	Faible / Élevé
Moyen / Faible	Moyen / Moyen	Moyen / Élevé
Élevé / Faible	Élevé / Moyen	Élevé / Élevé

Matrice impact × probabilité pour prioriser les risques techniques.

Exercice 386 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 387 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 388 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 389 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 390 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 391 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 392 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 393 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 394 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 395 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 396 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 397 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 398 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 415 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 416 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 417 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 418 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 419 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Exercice 420 : analysez un problème lié à Projets complets. Produisez un schéma, une proposition de structure de dossiers, une règle de nommage, un test minimal et une note de risque. Corrigé attendu : séparation claire des responsabilités, dépendances explicites, comportement testable et documentation courte.

Examens blancs

Examen blanc 1

- Question 1 : analyser une architecture existante et identifier trois faiblesses.
- Question 2 : proposer une arborescence professionnelle.
- Question 3 : écrire un service métier testable.
- Question 4 : produire un diagramme de flux.
- Question 5 : documenter une décision d'architecture.
- Projet : présenter une solution complète devant un jury.

Examen blanc 2

- Question 1 : analyser une architecture existante et identifier trois faiblesses.
- Question 2 : proposer une arborescence professionnelle.
- Question 3 : écrire un service métier testable.
- Question 4 : produire un diagramme de flux.
- Question 5 : documenter une décision d'architecture.
- Projet : présenter une solution complète devant un jury.

Examen blanc 3

- Question 1 : analyser une architecture existante et identifier trois faiblesses.
- Question 2 : proposer une arborescence professionnelle.
- Question 3 : écrire un service métier testable.
- Question 4 : produire un diagramme de flux.
- Question 5 : documenter une décision d'architecture.
- Projet : présenter une solution complète devant un jury.

Examen blanc 4

- Question 1 : analyser une architecture existante et identifier trois faiblesses.
- Question 2 : proposer une arborescence professionnelle.

- Question 3 : écrire un service métier testable.
- Question 4 : produire un diagramme de flux.
- Question 5 : documenter une décision d'architecture.
- Projet : présenter une solution complète devant un jury.

Examen blanc 5

- Question 1 : analyser une architecture existante et identifier trois faiblesses.
- Question 2 : proposer une arborescence professionnelle.
- Question 3 : écrire un service métier testable.
- Question 4 : produire un diagramme de flux.
- Question 5 : documenter une décision d'architecture.
- Projet : présenter une solution complète devant un jury.

Examen blanc 6

- Question 1 : analyser une architecture existante et identifier trois faiblesses.
- Question 2 : proposer une arborescence professionnelle.
- Question 3 : écrire un service métier testable.
- Question 4 : produire un diagramme de flux.
- Question 5 : documenter une décision d'architecture.
- Projet : présenter une solution complète devant un jury.

Glossaire professionnel

Terme	Définition
Architecture logicielle	Organisation des composants, responsabilités, flux, règles et contraintes d'un système.
Module	Unité cohérente regroupant une responsabilité claire.
Service	Composant qui orchestre un cas d'usage ou une opération métier.
MVC	Modèle de séparation entre données, interface et contrôle.
Clean code	Code lisible, intentionnel, testable et maintenable.
ADR	Architecture Decision Record, note courte expliquant une décision technique.
Observabilité	Capacité à comprendre l'état du système via logs, métriques et traces.
Security by design	Intégration de la sécurité dès la conception.

Feuille de route pour devenir architecte logiciel

- Maîtriser un langage et écrire du code propre.
- Comprendre les bases de données, API, tests et sécurité.
- Savoir modéliser avec des diagrammes simples.
- Lire du code existant et proposer des refactorings progressifs.
- Documenter les décisions et expliquer les compromis.
- Participer à des revues de code et incidents de production.
- Concevoir un système complet, puis l'exploiter en conditions réelles.

Contact professionnel

Charmant Nyungu — Consultant en innovation technologique, transformation numérique, cybersécurité, intelligence artificielle, développement logiciel et stratégie digitale.

- www.charmantnyungu.com
- consultant@charmantnyungu.com
- +243 835 137 837